



Lambda & Functional Programming

LLD-11 • Python Advanced-3

Functions are objects. Once you see that, **lambda**, **map**, **filter**, **closures**, and **decorators** all click.

RECAP

Quick Check — Collections

Before we move on, let's make sure last class stuck.

What happens here?

```
from collections import defaultdict

d = defaultdict(list)
d["existing"].append(1)

result = d["new_key"]
print(result)
```

What does `d["new_key"]` return?

KeyError

None

[] — an empty list

0

deque with maxlen

LLD-11: Lambda & FP

```
q = deque(maxlen=3)
for i in range(1, 6): # 1, 2, 3, 4, 5
    q.append(i)

print(q)
```

After appending 1, 2, 3, 4, 5 to a deque(maxlen=3), what's left?

deque([1, 2, 3])

deque([3, 4, 5])

deque([1, 2, 3, 4, 5])

Error — deque is full

```
Counter

from collections import Counter

c = Counter("mississippi")
print(c['s'])
```

What is c['s']?

2

3

LLD-11: Lambda & FP

5

THE FOUNDATION

Functions Are Objects

This one idea unlocks everything else in this class. Take it slow.

THINK ABOUT IT

What IS a function, really? In Python, a function is not some magical special thing. It's just an **object** — like an int, a string, or a list. You can store it, pass it, return it.

1 A function IS an object

type() proves it

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(type(greet))  
# <class 'function'>
```

LLD-11: Lambda & FP

Both are objects. Both have a type. Both live in memory.

2 Store a function in a variable

● ● ● No parentheses = the function itself

```
def greet(name):  
    return f"Hello, {name}!"  
  
say_hi = greet          # No parentheses! Storing the function object  
print(say_hi("Vipul")) # "Hello, Vipul!"  
  
# say_hi and greet point to the SAME function object  
print(say_hi is greet) # True
```

What's the difference between greet and greet()?

They're the same thing

greet is the function object itself; greet() CALLS it and gives the return value

greet returns the function name as a string

greet() is the function object; greet calls it

3 Pass a function as an argument

● ● ● Higher-order function

LLD-11: Lambda & FP

```
def subtract(a, b):
    return a - b

def apply(func, x, y):
    """Takes a function and two numbers, applies the function."""
    return func(x, y)

print(apply(add, 5, 3))      # 8
print(apply(subtract, 5, 3)) # 2

# apply() is a "higher-order function" – it takes a function as input
```

4 Return a function from a function

```
Function factory ▼

def make_greeter(greeting):
    """Returns a NEW function that uses the greeting."""
    def greeter(name):
        return f"{greeting}, {name}!"
    return greeter # Return the function object (no parentheses!)

hello = make_greeter("Hello")
hola = make_greeter("Hola")

print(hello("Vipul")) # "Hello, Vipul!"
print(hola("Karthik")) # "Hola, Karthik!"

# Each call to make_greeter() creates a brand new function
```



Another example: Discount calculator factory



LLD-11: Lambda & FP

Everything in the rest of this class builds on these 4 steps.

ANONYMOUS FUNCTIONS

Lambda — A Function Without a Name

Sometimes you need a tiny function for just one line. Lambda gives you that.

THINK ABOUT IT

Problem: You want to sort a list of dicts by age. You need a function that extracts the age. Do you really need a whole `def` for `return person["age"]`?

def vs lambda — side by side

Regular def

```
2 lines
def double(x):
    return x * 2
```

LLD-11: Lambda & FP

Lambda equivalent

1 expression

```
double = lambda x: x * 2  
  
print(double(5)) # 10
```

Building the syntax step by step

Lambda variations

```
# One parameter  
lambda x: x * 2  
# "given x, return x * 2"  
  
# Two parameters  
lambda a, b: a + b  
# "given a and b, return a + b"  
  
# No parameters  
lambda: 3.14  
# "takes nothing, returns 3.14"  
  
# With a ternary (conditional)  
lambda x: "even" if x % 2 == 0 else "odd"  
# "given x, return 'even' or 'odd'"
```

Lambda syntax breakdown

LLD-11: Lambda & FP

x, y
parameters

:
separator

x + y
expression (returned)

No return keyword. No function name. One expression only.

What's wrong with this code?

```
# Is this good Python?  
double = lambda x: x * 2  
print(double(5)) # 10
```

Nothing — it's perfectly fine

Lambda can't take parameters

It's an anti-pattern — if you're naming it, just use def

Lambda is slower than def

Where lambda actually shines

LLD-11: Lambda & FP

Lambda (concise)

One-liner

```
students = [
    {"name": "Vipul", "age": 22},
    {"name": "Karthik", "age": 20},
    {"name": "Ajit", "age": 25},
]

sorted(students, key=lambda s: s["age"])
# Karthik(20), Vipul(22), Ajit(25)
```

Without lambda

More code for same result

```
def get_age(student):
    return student["age"]

sorted(students, key=get_age)
# Same result – but we created a whole
# function just to extract one field
```

2. Finding max/min by a key

Lambda (concise)

Quick inline

LLD-11: Lambda & FP

```
shortest_name = min(students, key=lambda s: len(s["name"]))  
# {'name': 'Ajit', 'age': 25} (4 chars)
```

Without lambda

● ● ● Verbose alternative

```
def get_age(student):  
    return student["age"]  
  
def get_name_length(student):  
    return len(student["name"])  
  
oldest = max(students, key=get_age)  
shortest_name = min(students, key=get_name_length)  
# 2 extra functions just for one-time use
```

3. Dictionary of operations (mini calculator)

Lambda (concise)

● ● ● Compact dispatch table

```
ops = {  
    "+": lambda a, b: a + b,  
    "-": lambda a, b: a - b,  
    "*": lambda a, b: a * b,  
}  
print(ops["+"](3, 4)) # 7
```

LLD-11: Lambda & FP

OOP approach

```
Class-based calculator

class Calculator:
    def add(self, a, b): return a + b
    def sub(self, a, b): return a - b
    def mul(self, a, b): return a * b

    def compute(self, op, a, b):
        if op == "+": return self.add(a, b)
        if op == "-": return self.sub(a, b)
        if op == "*": return self.mul(a, b)

calc = Calculator()
print(calc.compute("+", 3, 4)) # 7
# Works, but way more code for a simple lookup
```

INTERVIEW READY

Lambda rules to remember:

- **One expression only** — no statements, no assignments, no loops
- **No name** — that's the point. Use inline with `sorted()`, `map()`, `filter()`
- **Implicit return** — the expression's value IS the return value
- **Use `def` if:** the function needs a name, docstring, multiple lines, or appears in tracebacks

You need a function to validate email addresses with regex, logging, and error messages. Lambda or `def`?

Lambda — keep it short

LLD-11: Lambda & FP

You're sorting a list of tuples `[(3, 'c'), (1, 'a'), (2, 'b')]` by the second element. Lambda or def?

Lambda — `sorted(data, key=lambda x: x[1])`

def — define a `get_second_element` function first

BEFORE WE CONTINUE

Iterable vs Iterator

You'll see the words "iterable" and "iterator" everywhere in `map()`, `filter()`, and `reduce()`. Let's clear this up first.

THINK ABOUT IT

Quick question: When you write `for x in [1, 2, 3]`, what is `[1, 2, 3]`? And what is Python doing behind the scenes to give you one item at a time?

LLD-11: Lambda & FP

contains pages, and you *can* read through it.

● ● ● These are all iterables

```
# Lists, tuples, strings, dicts, sets, files – all iterable
for x in [1, 2, 3]:          # list ✓ → loops over elements
    pass
for x in (1, 2, 3):         # tuple ✓ → loops over elements
    pass
for ch in "hello":          # string ✓ → loops over each character
    pass

# Yes, dict and set are iterable too!
d = {"name": "Vipul", "psp": 85}
for k in d:                 # dict ✓ → loops over KEYS ("name", "psp")
    pass
for k, v in d.items():      # dict.items() → loops over (key, value) pairs
    pass

s = {10, 20, 30}
for x in s:                 # set ✓ → loops over elements (unordered!)
    pass

for n in range(5):          # range ✓ → loops over 0, 1, 2, 3, 4
    pass

# An iterable knows HOW to create an iterator
# Think: "I'm a book – ask me and I'll give you a bookmark"
```

Which of these can you **NOT** loop over with `for x in ...?`

[1, 2, 3]

LLD-11: Lambda & FP

42

range(10)

Iterator = "I *am* looping — and I remember where I am"

An **iterator** is the object that actually does the work of producing values **one at a time**. It's the **bookmark** that keeps track of which page you're on.

Iterables create iterators

```
nums = [10, 20, 30]           # This is an ITERABLE (the book)

it = iter(nums)               # This creates an ITERATOR (the bookmark)

print(next(it))               # 10  - first value
print(next(it))               # 20  - remembers position, gives next
print(next(it))               # 30  - next value
print(next(it))               # StopIteration error - no more items!

# A for loop does exactly this behind the scenes:
# 1. Calls iter() on the iterable to get an iterator
# 2. Calls next() repeatedly until StopIteration
```

Iterable vs Iterator

Iterable

[1, 2, 3], "hello", range(5)



LLD-11: Lambda & FP



Iterator

produces values one by one

Iterable = the data source (book). Iterator = the cursor walking through it (bookmark).
You can create **multiple iterators** from one iterable — like multiple bookmarks in the same book.

Why does this matter?

`map()`, `filter()`, and `range()` all return **iterators**, not lists. That means:

● ● ● Lazy = memory efficient

```
# This does NOT create a list of 10 million numbers in memory
result = map(lambda x: x * 2, range(10_000_000))
# result is an iterator – nothing computed yet!

print(next(result)) # 0 – computes just this one
print(next(result)) # 2 – computes just this one

# To get ALL values at once, wrap in list():
# all_values = list(result) – but careful with 10M items!

# Iterator = lazy (one at a time, saves memory)
# List      = eager (all at once, uses memory)
```

Which of these is an iterator (not just an iterable)?

LLD-11: Lambda & FP

`"hello"` — a string

`map(lambda x: x*2, [1,2,3])` — a map object

`{1, 2, 3}` — a set

TL;DR: An **iterable** is anything you can loop over (`list`, `str`, `dict`, `range`). An **iterator** is the lazy worker that gives you values one at a time (`map()`, `filter()`, `iter()` result). Iterators save memory because they don't store everything upfront. When you see `map()` and `filter()` returning iterators — now you know why.



A bit of history: `map/filter` vs comprehensions



TRANSFORM

`map()` — Transform Every Element

Apply a function to every item in an iterable. Like a conveyor belt.

LLD-11: Lambda & FP

Returns: an **iterator** (lazy) — not a list! Wrap in `list()` to see all values

What it does: applies the function to each element, one at a time

map() — Conveyor Belt

1

2

3

4



$f(x) = x * 2$
the function



2

4

6

8

LLD-11: Lambda & FP

THINK ABOUT IT

Problem: You have prices in dollars: [10, 20, 30, 40]. Convert them all to euros (multiply by 0.85). How many ways can you do it?

Approach 1: for loop

● ● ● The explicit way

```
prices = [10, 20, 30, 40]
euros = []
for p in prices:
    euros.append(p * 0.85)
# [8.5, 17.0, 25.5, 34.0]
```

Approach 2: list comprehension

● ● ● Pythonic

```
euros = [p * 0.85 for p in prices]
# [8.5, 17.0, 25.5, 34.0]
```

What is a list comprehension? It's Python's shortcut for "build a new list by transforming each item": [expression for item in iterable]. It reads like English: *"give me p * 0.85 for each p in prices"*. Comprehensions are considered more **Pythonic** (idiomatic Python) than `map()` because they're easier to read at a glance.

Approach 3: map()

LLD-11: Lambda & FP

```
euros = list(map(lambda p: p * 0.85, prices))  
# [8.5, 17.0, 25.5, 34.0]
```

What is an Iterator?

An **iterator** is an object that produces values **one at a time**, on demand — instead of storing everything in memory at once.

You already use iterators without realizing it:

 Iterators you already know

```
# range() is an iterator — it doesn't create a list of 1 million numbers  
for i in range(1_000_000): # generates one number at a time  
    pass  
  
# file reading is an iterator — reads one line at a time  
for line in open("data.txt"): # doesn't load entire file into memory  
    pass  
  
# enumerate() is an iterator  
for i, val in enumerate(["a", "b", "c"]):  
    pass  
  
# map() and filter() also return iterators!
```

Why lazy? Imagine processing 10 million rows from a database. A list loads ALL 10M rows into memory. An iterator processes them **one at a time** — constant memory, no matter the size.

map() returns...

LLD-11: Lambda & FP

An iterator (lazy) — you need `list()` to see the values

A tuple

A generator function

`map()` with a named function — `to_celsius`



map vs comprehension — comprehension usually wins in Python.

Comprehensions are more readable, more **Pythonic** (meaning: they follow Python's philosophy of "readability counts"), and just as fast.

Use `map()` when:

- You already have a named function: `map(str.upper, words)` — cleaner than `[s.upper() for s in words]`
- You're chaining with other FP tools
- You want lazy evaluation without writing a generator expression

Use comprehension when:

- The transformation is a simple expression
- You also need filtering: `[x*2 for x in nums if x > 0]`
- You want readability (90% of cases)

SELECT

filter() — Keep Only What Matches

Test each element. Keep it if True, discard if False.

Syntax: `filter(function, iterable)`

Takes: a function that returns True/False (a "predicate"), and an iterable

Returns: an **iterator** (lazy) — just like `map()`, wrap in `list()` to see values

What it does: keeps only elements where the function returns True

filter() — The Gatekeeper

-3

7

-1

4

0

→

`x > 0 ?`

LLD-11: Lambda & FP

7

4

Items that pass the test stay. Items that fail are dropped.

THINK ABOUT IT

Problem: Given `nums = [-3, 7, -1, 4, 0, 12, -8, 5]`, keep only the positive numbers.

Loop vs comprehension vs filter

```
nums = [-3, 7, -1, 4, 0, 12, -8, 5]

# 1. For loop
positives = []
for n in nums:
    if n > 0:
        positives.append(n)

# 2. List comprehension (most Pythonic)
positives = [n for n in nums if n > 0]

# 3. filter()
positives = list(filter(lambda n: n > 0, nums))

# All produce: [7, 4, 12, 5]
```

LLD-11: Lambda & FP

Clean up messy data

```
messy = ["hello", "", "world", None, "foo", 0, "bar", False]

clean = list(filter(None, messy))
# ["hello", "world", "foo", "bar"]

# filter(None, iterable) keeps only TRUTHY values
# Removes: "", None, 0, False, [], {}
```



Real-world: Filter students by PSP (Problem Solving Points)



Which is more readable for "get names of students with PSP $\geq$ 70"?

```
# Option A: filter + map
list(map(lambda s: s["name"], filter(lambda s: s["psp"]  $\geq$  70, students)))

# Option B: comprehension
[s["name"] for s in students if s["psp"]  $\geq$  70]
```

Option A — filter + map is more explicit

Option B — comprehension reads like English

They're equally readable

ACCUMULATE

reduce() — Combine Into One

Take a list of things, boil them down to a single value.

Syntax: `reduce(function, iterable[, initial])`

Takes: a function with **two parameters** (accumulator, current_item), an iterable, and an optional initial value

Returns: a **single value** — the final accumulated result

What it does: applies the function cumulatively — takes first two items, combines them, then combines result with next item, and so on

Import: `from functools import reduce` — it's NOT a built-in

reduce(+, [1, 2, 3, 4, 5]) — Step by Step

1

2

→

3

3

LLD-11: Lambda & FP

→

6

6

4

→

10

10

5

→

15

accumulator + next item → new accumulator. Repeat until one value remains.

● ● ● reduce examples ▼

```
from functools import reduce
```

```
# Sum
```

```
reduce(lambda acc, x: acc + x, [1, 2, 3, 4, 5])
```

```
# 15 (but just use sum(!))
```

```
# Product
```

LLD-11: Lambda & FP

```
# Find max (manually)
reduce(lambda acc, x: acc if acc > x else x, [3, 7, 2, 9, 1])
# 9 (but just use max()!)

# Flatten nested lists
reduce(lambda acc, x: acc + x, [[1,2], [3,4], [5,6]])
# [1, 2, 3, 4, 5, 6]
```

`reduce(lambda acc, x: acc * x, [1, 2, 3, 4, 5]) = ?`

15

120

60

Error — reduce needs an initial value

Note: reduce is in functools, not built-in. [Guido van Rossum](#) moved it out of built-ins because Python has better alternatives for most cases: `sum()`, `max()`, `min()`, `math.prod()`, `"".join()`. Use reduce only when no built-in does what you need.

THE PYTHON WAY

Comprehensions vs FP

Python is not Haskell. Know when each tool is the right choice.

The same task, two styles

FP style (map + filter + lambda)

```
Hard to read

# Get squared values of even numbers
result = list(
    map(
        lambda x: x ** 2,
        filter(
            lambda x: x % 2 == 0,
            numbers
        )
    )
)
```

Comprehension (Pythonic)

```
Clean and clear

# Get squared values of even numbers
```

LLD-11: Lambda & FP

When to use which

Tool	Best For	Example
List comprehension	Transform + filter in one step	<code>[x**2 for x in nums if x > 0]</code>
Dict comprehension	Build dicts from iterables	<code>{k: v for k, v in pairs}</code>
Set comprehension	Unique transformed values	<code>{word.lower() for word in words}</code>
Generator expression	Lazy, memory-efficient pipelines	<code>sum(x**2 for x in nums)</code>
map()	Existing named function, lazy eval	<code>map(str.upper, words)</code>
filter()	Existing predicate function	<code>filter(str.isdigit, chars)</code>
reduce()	Cumulative combination, no built-in	<code>reduce(operator.mul, nums)</code>

Rewrite `list(map(lambda x: x.upper(), words))` as a comprehension:

```
[x.upper() for x in words]
```

LLD-11: Lambda & FP

```
[words.upper() for x in words]
```

```
{x.upper() for x in words}
```

FUNCTIONS THAT REMEMBER

Closures

When a function remembers variables from its enclosing scope, even after that scope is gone.

THINK ABOUT IT

Question: In `make_greeter("Hello")`, the parameter "Hello" should vanish when `make_greeter` finishes. But the returned function still uses it. How?

1 Inner function accesses outer variable

Just scope rules

```
def outer():  
    message = "Hello" # outer variable
```

LLD-11: Lambda & FP

```
    inner() # "Hello"

outer()
```

2 Return the inner function — it REMEMBERS

```
● ● ● This is a closure ▼

def make_greeter(greeting):
    def greeter(name):
        return f"{greeting}, {name}!" # "greeting" comes from outer scope
    return greeter # Return the function. outer scope is now "gone".

hello = make_greeter("Hello")
# make_greeter has finished executing.
# But "hello" still remembers greeting = "Hello"!

print(hello("Vipul")) # "Hello, Vipul!"
print(hello("Kaarthik")) # "Hello, Kaarthik!"
```

3 Each closure has its own state

```
● ● ● make_counter — private state ▼

def make_counter(start=0):
    count = [start] # Use a list so inner function can modify it

    def counter():
        count[0] += 1
        return count[0]

    return counter
```

LLD-11: Lambda & FP

```
print(c1()) # 1
print(c1()) # 2
print(c2()) # 101
print(c1()) # 3 — c1 and c2 are independent!
```

4 Closures with lambda

```
● ● ● make_multiplier ▼

def make_multiplier(factor):
    return lambda x: x * factor # lambda closes over "factor"

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5)) # 10
print(triple(5)) # 15
```

After `hello = make_greeter("Hello")`, what does `hello("Vipul")` return?

Error — greeting is out of scope

"Hello, Vipul!"

None

"Vipul"

LLD-11: Lambda & FP

Bug: [4, 4, 4, 4, 4]

What you'd expect to work

```
funcs = []
for i in range(5):
    funcs.append(lambda: i)

# You'd expect [0, 1, 2, 3, 4]
print([f() for f in funcs])
# [4, 4, 4, 4, 4] ← BUG!
```

Fix: capture the value

Default argument trick

```
funcs = []
for i in range(5):
    funcs.append(lambda i=i: i)
    #           ^^^ capture current value!

print([f() for f in funcs])
# [0, 1, 2, 3, 4] ← CORRECT
```

Lambda captures the VARIABLE, not the VALUE. All 5 lambdas share the same variable `i`. By the time you call them, the loop is done and `i = 4`. The fix: `lambda i=i: i` evaluates `i` at definition time and stores it as a default argument. This is a classic Python interview gotcha.

CLOSURES IN DISGUISE

Decorators Preview

Decorators are just closures with syntactic sugar. Once you see it, they're not magic anymore.

THINK ABOUT IT

Problem: You want to measure how long a function takes. You could add timing code inside every function... or you could **wrap** the function.

1 A function that wraps another function

```
timer decorator (manual)

import time

def timer(func):
    """Takes a function, returns a new function that times it."""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)    # Call the original
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f}s")
        return result
    return wrapper
```

LLD-11: Lambda & FP

```
# Manually wrap it:
calculate = timer(calculate)
calculate()
# "calculate took 0.0312s"
```

2 @timer is just syntactic sugar

● ● ● The @ syntax ▼

```
@timer                                # This is identical to: calculate = timer(calculate)
def calculate():
    return sum(range(1_000_000))

calculate()
# "calculate took 0.0312s"

# @decorator is NOT magic. It's just:
# function = decorator(function)
```



Another decorator: log_calls ▼

What is @timer equivalent to?

timer.calculate()

calculate = timer(calculate)

timer = calculate(timer)

LLD-11: Lambda & FP

INTERVIEW READY

Real decorators you'll use in production:

- **@functools.cache** — memoize expensive function calls (cache results)
- **@login_required** — Django: block unauthenticated users
- **@app.route("/path")** — Flask/FastAPI: register URL routes
- **@property** — make a method behave like an attribute
- **@staticmethod, @classmethod** — OOP method types
- **@pytest.fixture** — test setup/teardown

DECISION GUIDE

FP vs OOP — When to Use Which

The same problem solved three ways. Then a guide for choosing.

THINK ABOUT IT

Problem: Apply a 10% discount to all prices over \$100 and get the total.

● ● ● OOP approach ▼

LLD-11: Lambda & FP

```
self.prices = prices

def apply_discount(self, threshold=100, discount=0.10):
    return [
        p * (1 - discount) if p > threshold else p
        for p in self.prices
    ]

def total(self):
    return sum(self.apply_discount())

calc = PriceCalculator([50, 150, 200, 80])
print(calc.total()) # 445.0
```

● ● ● FP approach ▼

```
from functools import reduce

prices = [50, 150, 200, 80]

discounted = map(lambda p: p * 0.9 if p > 100 else p, prices)
total = reduce(lambda acc, p: acc + p, discounted)
print(total) # 445.0
```

● ● ● Pythonic approach (mix both) ▼

```
prices = [50, 150, 200, 80]

total = sum(p * 0.9 if p > 100 else p for p in prices)
print(total) # 445.0
```

Decision table

LLD-11: Lambda & FP

Transform data (map, filter, sort)	FP / Comprehensions	Stateless, one-liners, readable pipelines
Manage state (user sessions, DB connections)	OOP	Objects encapsulate state and behavior together
One-off data pipeline	Comprehension + built-ins	No need for classes or imports
Complex business logic with rules	OOP	Methods, inheritance, polymorphism
Config, callbacks, event handlers	FP (closures, lambdas)	Small, composable, no class overhead
Django views, models, serializers	OOP + FP mix	Classes for structure, comprehensions for querysets

Concrete examples: Pick the right tool



Example 1: Get all student names from a list of dicts



Example 2: Track an order through its lifecycle



Example 3: Process sales data — filter, transform, aggregate



Example 4: Django-style — OOP structure + FP data transforms



LLD-11: Lambda & FP

everything into one paradigm. The best Python code uses the right tool for each job.

QUICK REFERENCE

map vs filter vs reduce — At a Glance

Click each cell to reveal the answer. Test yourself first!

	map()	filter()	reduce()
Purpose	click to reveal	click to reveal	click to reveal
Input	click to reveal	click to reveal	click to reveal
Output	click to reveal	click to reveal	click to reveal
Lazy?	click to reveal	click to reveal	click to reveal
Built-in?	click to reveal	click to reveal	click to reveal
Example	click to reveal	click to reveal	click to reveal

LLD-11: Lambda & FP

Think of it
as

click to reveal

click to reveal

click to reveal

Visual summary

map(f, [a, b, c])

↓

[f(a), f(b), f(c)]

Same count, new values

filter(f, [a, b, c])

↓

[a, c] (if f(a), f(c) = True)

Fewer items, same values

reduce(f, [a, b, c])

↓

f(f(a, b), c) → one value

Everything → single result

PRACTICE

LLD-11: Lambda & FP

Try each one yourself before revealing the solution. Use map, filter, reduce as indicated.

Exercise 1: map() — Add 18% GST to all prices

```
prices = [1200, 450, 3200, 890, 150]

# Use map() to multiply each price by 1.18
with_gst = ???
```



Reveal Solution



Exercise 2: filter() — Students with attendance $\geq$ 85%

```
students = [
    {"name": "Vipul", "psp": 85, "attendance": 92},
    {"name": "Karthik", "psp": 92, "attendance": 88},
    {"name": "Ajit", "psp": 78, "attendance": 95},
    {"name": "Gobi", "psp": 45, "attendance": 60},
    {"name": "Sneha", "psp": 91, "attendance": 85},
]

# Use filter() to keep only students with attendance  $\geq$  85
regular = ???
```

LLD-11: Lambda & FP

Exercise 3: reduce() — Total PSP of all students

```
from functools import reduce

# Use reduce() to sum up all PSP values from the students list above
total_psp = ???
```

💡 **Reveal Solution**



Exercise 4: filter + map — Get NAMES of students with PSP $\geq$ 70

```
# Step 1: filter students where PSP  $\geq$  70
# Step 2: map to extract just the name
# Do it with filter+map, then try the comprehension way
strong_names = ???
```

💡 **Reveal Solution**



Exercise 5: map + reduce — Total revenue after 18% GST

LLD-11: Lambda & FP

```
# Step 1: map each price to price * 1.18 (add GST)
# Step 2: reduce to get the total
total_revenue = ???
```



Reveal Solution



Exercise 6: filter + map + reduce — Total PSP of students with attendance $\geq$ 80%

```
# Combine all three!
# Step 1: filter — keep students with attendance  $\geq$  80
# Step 2: map — extract their PSP values
# Step 3: reduce — sum them up
# Then write the Pythonic version too
total_psp_regular = ???
```



Reveal Solution



Exercise 7: sorted + lambda — Rank students by PSP (highest first)

```
# Sort the students list by PSP in descending order
# Print each with their rank: #1 Kaarthik: PSP=92, ...
ranked = ???
```

DEEP DIVE

Comprehensions — The Full Picture

We've seen comprehensions beat map/filter all class. Let's master the syntax — from basic to advanced.

The anatomy of a list comprehension

Syntax breakdown

[

expression

what to produce

for item in iterable

where to get data

if condition

LLD-11: Lambda & FP

J

Read it like English: "give me *expression* for each *item* in *iterable* if *condition*"

1 Basic — Transform every element

```
[expression for item in iterable]
```

```
# Square every number
nums = [1, 2, 3, 4, 5]
squares = [n ** 2 for n in nums]
# [1, 4, 9, 16, 25]

# Convert to uppercase
names = ["vipul", "kaarthik", "ajit"]
upper = [name.upper() for name in names]
# ['VIPUL', 'KAARTHIK', 'AJIT']

# Add GST to prices
prices = [100, 250, 500]
with_gst = [p * 1.18 for p in prices]
# [118.0, 295.0, 590.0]

# Equivalent to: list(map(lambda n: n**2, nums))
# But more readable!
```

2 With filter — Keep only matching elements

```
[expression for item in iterable if condition]
```

```
# Only even numbers
evens = [n for n in range(1, 11) if n % 2 == 0]
# [2, 4, 6, 8, 10]
```

LLD-11: Lambda & FP

```
{ "name": "Vipul", "psp": 85 },
{ "name": "Gobi", "psp": 45 },
{ "name": "Karthik", "psp": 92 },
]
strong = [s["name"] for s in students if s["psp"] >= 70]
# ['Vipul', 'Karthik']

# Filter + transform in ONE line
# "give me name for each student if PSP >= 70"

# Equivalent to:
# list(map(lambda s: s["name"], filter(lambda s: s["psp"] >= 70, students)))
# Which one would you rather read? 🤔
```

3 With ternary — Transform differently based on condition

Don't confuse `if` at the END (filter) vs `if/else` in the EXPRESSION (ternary).

- `[x for x in nums if x > 0]` — filter: keeps only positives, drops the rest
- `[x if x > 0 else 0 for x in nums]` — ternary: keeps ALL items, replaces negatives with 0

● ● ● `[expr_if_true if condition else expr_if_false for item in iterable]` ▼

```
# Label each number
nums = [1, -3, 5, -7, 10]
labels = ["positive" if n > 0 else "negative" for n in nums]
# ['positive', 'negative', 'positive', 'negative', 'positive']

# Replace negatives with zero
cleaned = [n if n > 0 else 0 for n in nums]
```

LLD-11: Lambda & FP

```
results = [  
    f"{s['name']}: Pass" if s["psp"] ≥ 70 else f"{s['name']}: Needs Improvement"  
    for s in students  
]  
# ['Vipul: Pass', 'Gobi: Needs Improvement', 'Karthik: Pass']
```

What's the difference between these two?

```
# Version A:  
[x ** 2 for x in nums if x > 0]  
  
# Version B:  
[x ** 2 if x > 0 else 0 for x in nums]
```

They're the same thing

A filters out non-positives (fewer items); B replaces them with 0 (same count)

A is invalid syntax

4 Nested loops — Multiple for clauses

```
● ● ● [expression for x in iter1 for y in iter2] ▼  
  
# All combinations of two lists  
colors = ["red", "blue"]  
sizes = ["S", "M", "L"]  
  
combos = [f"{c}-{s}" for c in colors for s in sizes]  
# ['red-S', 'red-M', 'red-L', 'blue-S', 'blue-M', 'blue-L']
```

LLD-11: Lambda & FP

```
#     for s in sizes:
#         combos.append(f"{c}-{s}")
```

● ● ● Flatten a nested list ▼

```
# Flatten [[1,2], [3,4], [5,6]] → [1, 2, 3, 4, 5, 6]
nested = [[1, 2], [3, 4], [5, 6]]

flat = [item for sublist in nested for item in sublist]
# [1, 2, 3, 4, 5, 6]

# Read order: outer for first, then inner for
# "for each sublist in nested, for each item in sublist, give me item"

# Equivalent to:
# flat = []
# for sublist in nested:
#     for item in sublist:
#         flat.append(item)
```

5 Dict & Set comprehensions

Same syntax, different brackets: {} for dict/set instead of [] for list.

● ● ● Dict comprehension: {key: value for item in iterable} ▼

```
# Student name → PSP lookup
students = [
    {"name": "Vipul", "psp": 85},
    {"name": "Kaarthik", "psp": 92},
    {"name": "Ajit", "psp": 78},
]

name_to_psp = {s["name"]: s["psp"] for s in students}
```


LLD-11: Lambda & FP

```
strong_lookup = {s["name"]: s["psp"] for s in students if s["psp"] ≥ 80}
# {'Vipul': 85, 'Karthik': 92}

# Swap keys and values
psp_to_name = {v: k for k, v in name_to_psp.items()}
# {85: 'Vipul', 92: 'Karthik', 78: 'Ajit'}
```

● ● ● Set comprehension: {expression for item in iterable} ▼

```
# Unique first letters
names = ["Vipul", "Karthik", "Ajit", "Gobi", "Sneha", "Kavya"]
first_letters = {name[0] for name in names}
# {'V', 'K', 'A', 'G', 'S'} – 'K' appears once even though 2 names start with K

# Unique word lengths in a sentence
sentence = "the quick brown fox jumps over the lazy dog"
lengths = {len(word) for word in sentence.split()}
# {3, 4, 5} – only unique lengths
```

6 Generator expression — Lazy comprehension

Replace `[]` with `()` and you get a **generator** — an iterator that computes values one at a time instead of building the whole list in memory.

● ● ● (expression for item in iterable) — lazy! ▼

```
# List comprehension – builds entire list in memory
total = sum([x ** 2 for x in range(1_000_000)]) # 1M items in memory!

# Generator expression – computes one at a time
total = sum(x ** 2 for x in range(1_000_000)) # almost zero memory!
#           ^ no brackets – that makes it a generator

# Use generators when:
```

Comprehension cheat sheet

Type	Syntax	Result
List	<code>[expr for x in iter]</code>	click to reveal
List + filter	<code>[expr for x in iter if cond]</code>	click to reveal
List + ternary	<code>[a if cond else b for x in iter]</code>	click to reveal
Nested	<code>[expr for x in iter1 for y in iter2]</code>	click to reveal
Dict	<code>{k: v for x in iter}</code>	click to reveal
Set	<code>{expr for x in iter}</code>	click to reveal
Generator	<code>(expr for x in iter)</code>	click to reveal

When NOT to use comprehensions:

- **More than 2 conditions or nested loops** — if you can't read it in 5 seconds, use a regular loop
- **Side effects** (printing, writing to file, modifying external state) — use a for loop instead
- **Complex multi-step logic** — break it into readable steps

```
# x BAD — too complex, nobody can read this
result = [f(x, y) for x in a if g(x) for y in b if h(x, y) if y != x]
```

LLD-11: Lambda & FP

```
for x in a:
    if not g(x):
        continue
    for y in b:
        if h(x, y) and y != x:
            result.append(f(x, y))
```

Convert this loop to a comprehension:

```
result = []
for s in students:
    if s["psp"] >= 70:
        result.append(s["name"].upper())
```

[s.upper() for s in students if s["psp"] >= 70]

[s["name"].upper() for s in students if s["psp"] >= 70]

{s["name"].upper() for s in students if s["psp"] >= 70}

What does this produce?

```
{s["name"]: "Pass" if s["psp"] >= 70 else "Fail" for s in students}
```

A list of "Pass"/"Fail" strings

A set of student names

WRAP UP

Key Takeaways



Functions as Objects

Functions can be stored in variables, passed as arguments, and returned from other functions. This is the foundation for everything else.



Lambda

Anonymous, one-expression functions. Use inline with `sorted(key=...)`, `map()`, `filter()`. Don't assign to a variable — use `def` instead.



map / filter / reduce

map transforms, **filter** selects, **reduce** combines. All return iterators



Closures

Inner functions that remember outer variables. Each closure gets its own

LLD-11: Lambda & FP



Decorators

`@decorator = func =
decorator(func)`. Just closures
with syntactic sugar. Used
everywhere: `@cache`,
`@login_required`, `@app.route`.

Next class: Python Advanced-4 — Exception Handling